

Relational Database

সিনিয়র লেভেল রিভিশন — ১৫ মিনিট। MySQL-কেন্দ্রিক, কারণ BD মার্কেটে বেশিরভাগ স্ট্যাক MySQL/MariaDB।

১. Index কীভাবে কাজ করে এবং কখন কাজ করে না?

Description: সবচেয়ে কমন সিনিয়র প্রশ্ন — শুধু "index দিলে fast হয়" নয়, B-Tree structure আর কোন কোন ক্ষেত্রে index skip হয় সেটা ব্যাখ্যা করতে পারতে হবে।

৬টা Key Point:

1. B-Tree index sorted structure — equality আর range query দুটোতেই কাজ করে; lookup $O(\log n)$
2. Composite index-এ leftmost prefix rule: (a, b, c) index $a, a+b, a+b+c$ query-তে লাগে, শুধু b দিয়ে খুঁজলে লাগে না
3. Index কাজ করে না: column-এ function ($DATE(created_at)$), leading wildcard ($LIKE '%abc'$), implicit type cast (string column-এ int compare)
4. Covering index: দরকারি সব column index-এই থাকলে টেবিলে যেতেই হয় না ($Using index$ in EXPLAIN) — সবচেয়ে দ্রুত
5. Low cardinality column-এ (যেমন status-এর ৩টা মান) একক index প্রায় অর্থহীন — composite-এর অংশ হিসেবে রাখা যায়
6. প্রতিটা index write-এর খরচ বাড়ায় — অপ্ৰয়োজনীয় index পরিষ্কার রাখা; EXPLAIN দিয়ে আসলেই ব্যবহার হচ্ছে কিনা দেখা

২. Transaction Isolation Level ও তাদের সমস্যা

Description: Concurrent transaction-এ কী কী anomaly হয় (dirty read, non-repeatable read, phantom) আর কোন isolation level কোনটা ঠেকায় — টাকা জড়িত সিস্টেমে এটা জানা বাধ্যতামূলক।

৬টা Key Point:

1. Read Uncommitted: dirty read সম্ভব (অন্যের uncommitted data দেখা) — কার্যত ব্যবহার হয় না
2. Read Committed: dirty read নেই, কিন্তু একই transaction-এ দুইবার পড়লে ভিন্ন ফল (non-repeatable read)
3. Repeatable Read (MySQL default): snapshot-এ consistent read; MySQL-এ gap lock দিয়ে phantom-ও প্রায় ঠেকায়
4. Serializable: সব anomaly বন্ধ, কিন্তু lock contention-এ throughput পড়ে যায় — বিশেষ ক্ষেত্র ছাড়া নয়
5. MVCC: reader-রা writer-কে block করে না — পুরনো version snapshot থেকে পড়ে; এজন্যই MySQL/Postgres-এ read এত সম্ভা
6. প্র্যাকটিক্যাল রুল: default Repeatable Read + দরকারি জায়গায় explicit $SELECT \dots FOR UPDATE$ — level বদলানোর আগে সমস্যাটা আসলে কী সেটা বোঝা

৩. Slow Query পেনে কীভাবে Debug ও Optimize করবেন?

Description: "একটা রিপোর্ট ৩০ সেকেন্ড নিচ্ছে" — এই বাস্তব পরিস্থিতিতে step-by-step কী করবেন, সেটার একটা পদ্ধতিগত উত্তর আশা করা হয়।

৬টা Key Point:

1. আগে ধরা: slow query log enable / APM — অনুমান নয়, কোন query কতবার কত সময় নিচ্ছে সেটা দেখা

2. **EXPLAIN** পড়া: type (ALL = full scan, খারাপ), rows (কত row scan), key (কোন index), Extra (Using filesort, Using temporary = লাল পতাকা)
3. সাধারণ fix অর্ডার: সঠিক index যোগ → **SELECT** * বাদ দিয়ে দরকারি column → JOIN কমানো/আগে filter করা
4. **OFFSET** বড় হলে pagination মরে যায় — keyset/cursor pagination (**WHERE id > last_id LIMIT n**)
5. Aggregation-heavy রিপোর্ট: pre-computed summary টেবিল / materialized রাখা — প্রতিবার লক্ষ row scan না করা
6. Query ঠিক থাকলে অন্য দিক দেখা: lock wait, connection pool শেষ, buffer pool ছোট — সমস্যা সবসময় query না-ও হতে পারে

8. Normalization vs Denormalization — কখন কোনটা?

Description: 3NF মুখস্থ বলা নয় — কোন বাস্তব পরিস্থিতিতে ইচ্ছা করে denormalize করবেন, সেই trade-off judgment-টাই প্রশ্ন।

৬টা Key Point:

1. Normalize করার মূল লাভ: update anomaly নেই, এক তথ্য এক জায়গায় — write-heavy transactional data সবসময় normalized রাখা
2. 3NF পর্যন্ত সাধারণত যথেষ্ট — এর বেশি academic, বাস্তবে দরকার পড়ে না
3. Denormalize করা হয় read performance-এর জন্য: বারবার JOIN-এর বদলে redundant column (যেমন order-এ **customer_name** snapshot)
4. Snapshot data আসলে denormalization না — invoice-এ তখনকার দাম রাখা business requirement (দাম পরে বদলালেও invoice ঠিক থাকবে)
5. Reporting-এর জন্য আলাদা denormalized summary টেবিল — transactional টেবিল normalized-ই থাকে; দুই জগৎ আলাদা রাখা
6. Denormalize করলে sync-এর দায়িত্ব নিজের: event/trigger/job দিয়ে redundant copy আপডেট রাখা — এই খরচটা স্বীকার করে সিদ্ধান্ত নেওয়া

৫. JOIN-এর ধরন ও Performance Behavior

Description: INNER/LEFT পার্থক্য বেসিক, সিনিয়রদের কাছে জানতে চাওয়া হয় JOIN internals (কীভাবে execute হয়) আর কখন JOIN এড়ানো উচিত।

৬টা Key Point:

1. INNER = দুই পাশেই match, LEFT = বাম পাশ পুরো + ডান পাশ NULL হতে পারে — LEFT JOIN + **WHERE right.id IS NULL** = "নেই এমন row" খোঁজা
2. MySQL nested loop join চালায় — ছোট (filtered) টেবিল আগে, join column-এ index না থাকলে বিপর্যয়
3. Join column দুই পাশে একই type + collation না হলে index skip হয় — খুব কমন গোপন বাগ
4. অনেক টেবিল join-এর চেয়ে মাঝে মাঝে দুইটা সরল query + app-level merge সস্তা (Eloquent eager loading আসলে এটাই করে)
5. Aggregate + join একসাথে করলে fan-out-এ ডাবল কাউন্টের ঝুঁকি — আগে subquery-তে aggregate, পরে join
6. Big table × big table join রিপোর্টে লাগলে সেটা design smell — summary টেবিল বা pre-aggregation-এর সংকেত

৬. Locking: Optimistic vs Pessimistic, Deadlock

Description: Concurrent write-এ ডেটা রক্ষার দুই কৌশল আর deadlock কেন হয়/কীভাবে এড়ায় — স্টক ও পেমেন্ট সিস্টেমের প্রাণকেন্দ্র।

৬টা Key Point:

1. Pessimistic: `SELECT ... FOR UPDATE` — আগে lock, পরে কাজ; conflict বেশি হলে (হট stock row) এটাই ঠিক
2. Optimistic: version/updated_at চেক করে `UPDATE ... WHERE version = X` — affected rows 0 হলে retry; conflict বিরল হলে বেশি throughput
3. Deadlock-এর মূল কারণ: দুই transaction ভিন্ন order-এ একই resource lock করে — সমাধান: সব জায়গায় একই order-এ (যেমন ID ascending) lock নেওয়া
4. MySQL deadlock detect করে একটাকে rollback করে — app-এ retry logic থাকা দরকার, error গিলে ফেলা নয়
5. Transaction ছোট রাখা: lock ধরে external API call/slow কাজ নয় — lock ধরার সময় যত কম, contention তত কম
6. Lock-free বিকল্প মনে রাখা: `atomic UPDATE stock SET qty = qty - 1 WHERE qty >= 1` — affected rows দেখে সিদ্ধান্ত; অনেক ক্ষেত্রে এটাই যথেষ্ট

৭. Database Scaling: Replication, Sharding, Partitioning

Description: এক সার্ভারে আর কুলাচ্ছে না — এরপর ধাপে ধাপে কী করবেন। প্রতিটা ধাপের trade-off জানা সিনিয়র হওয়ার লক্ষণ।

৬টা Key Point:

1. স্কেলিং অর্ডার: query/index optimize → বড় মেশিন (vertical) → read replica → cache → partition → sharding সবার শেষে
2. Read replica: async replication, তাই lag থাকে — "লিখেই সাথে সাথে পড়া" flow (write-then-read) primary থেকে পড়াতে হয়
3. Table partitioning (date/range অনুযায়ী): পুরনো ডেটা আলাদা partition-এ — prune হয়ে query দ্রুত, পুরনো partition drop সহজ
4. Sharding মানে data horizontal ভাগ (যেমন tenant অনুযায়ী) — cross-shard join/transaction হারাবেন; শেষ অস্ত্র
5. Multi-tenant SaaS-এ DB-per-tenant নিজেই একটা natural sharding — বড় tenant আলাদা সার্ভারে সরানো যায়
6. Connection pooling (ProxySQL/RDS Proxy) — অনেক app server × অনেক connection-এ DB-র মেমোরি শেষ হওয়া ঠেকায়

৮. SQL vs NoSQL — কখন কোনটা বেছে নেবেন?

Description: ধর্মযুদ্ধ নয়, workload-ভিত্তিক সিদ্ধান্ত। নিজের প্রজেক্টে দুটোই ব্যবহারের বাস্তব উদাহরণ দিতে পারলে উত্তর জোরালো হয়।

৬টা Key Point:

1. Relational data + ACID transaction + ad-hoc query = SQL; টাকা, অর্ডার, স্টক — সবসময় SQL
2. Document DB (MongoDB): flexible schema, nested data, aggregation pipeline — event/summary/log data-তে ভালো (Sokrio-তে delivery/order summary MongoDB-তে রাখি)
3. Key-value (Redis): cache, session, counter, queue — durability-critical data-র primary store নয়
4. NoSQL-এ "schema-less" মানে schema নেই তা নয় — schema app code-এ সরে যায়; সেটাও মেইনটেন করতে হয়
5. Polyglot বাস্তবতা: MySQL (transaction) + MongoDB (analytics/summary) + Redis (cache) একসাথে চলে — এক DB-তে সব জোর করা ভুল
6. সিদ্ধান্তের প্রশ্ন তিনটা: data relational কিনা, consistency requirement কী, query pattern আগে থেকে জানা কিনা

৯. Migration ও Schema Change বড় টেবিলে কীভাবে নিরাপদে করবেন?

Description: ৫ কোটি row-এর টেবিলে **ALTER TABLE** চালালে কী হয়? প্রোডাকশন অপারেশনের বাস্তব অভিজ্ঞতা যাচাইয়ের প্রশ্ন।

৬টা Key Point:

1. বড় টেবিলে সরাসরি ALTER = দীর্ঘ lock/metadata lock জ্যাম — পিক আওয়ারে চালালে ডাউনটাইম
2. MySQL 8-এ অনেক অপারেশন **ALGORITHM=INSTANT/INPLACE** — আগে চেক করা কোন algorithm লাগবে
3. Online tool: **pt-online-schema-change** / **gh-ost** — shadow টেবিল বানিয়ে copy + rename; lock প্রায় শূন্য
4. Backward-compatible ধাপে ভাঙা: আগে nullable column add → code deploy (দুটোই লেখে) → backfill job → পুরনো column পরে drop
5. Column drop/rename সবচেয়ে বিপজ্জনক — পুরনো code চলাকালীন ভাঙবে; দুই-ধাপ deploy বাধ্যতামূলক
6. Migration সবসময় rollback plan সহ — আর multi-tenant হলে সব tenant DB-তে লুপ করে চালানোর script + কোন tenant-এ fail হলো তার tracking

১০. ACID ও Real-world Consistency Trade-off

Description: ACID-এর সংজ্ঞা নয় — বাস্তবে কোথায় strict consistency দরকার আর কোথায় eventual consistency মেনে নেওয়া যায়, সেই বিচারবোধ।

৬টা Key Point:

1. Atomicity = সব বা কিছুই না; Consistency = নিয়ম ভাঙে না; Isolation = concurrent-এও সঠিক; Durability = commit মানে ডিস্কে টেকা
2. Strict consistency লাগে: ব্যালেন্স, স্টক decrement, পেমেন্ট state — এখানে transaction + lock, কোনো ছাড় নয়
3. Eventual চলে: dashboard কাউন্ট, রিপোর্ট, notification, search index — কয়েক সেকেন্ড পুরনো হলে ব্যবসার ক্ষতি নেই
4. Distributed transaction (2PC) বাস্তবে এড়ানো হয় — বদলে saga/outbox pattern: local transaction + event দিয়ে পরের ধাপ, ব্যর্থে compensating action
5. Outbox pattern মনে রাখা: DB write আর event publish এক transaction-এ (outbox টেবিলে), আলাদা worker event পাঠায় — dual-write সমস্যার সমাধান
6. ইন্টারভিউ ফ্রেমিং: "কোন ডেটার জন্য ইউজার ভুল দেখলে টাকা হারায়?" — সেটায় ACID, বাকিটা eventual + reconciliation